ЛАБОРАТОРНІ РОБОТИ № 6–7

РОЗРОБКА ФРОНТЕНДУ ПРОГРАМНОГО ПРОДУКТУ

Практична реалізація користувацького інтерфейсу
мобільного застосунку «Мої події» (Event Reminder)

Параметр	Значення
Предметна область	Управління особистими подіями та нагадуваннями
Цільова платформа	Android (Flutter), адаптивні макети desktop / mobile
Мова та фреймворк	Dart, Flutter 3.47.7, Material 3
Дані	SQLite, офлайн-персистентність
Статус документа	Технічний звіт за наявним вихідним кодом; місця для скриншотів реального UI потрібно заповнити

Підстава: методичні вказівки «Лабораторна робота № 6–7» та вихідний код event_reminder_complete.

1. Мета роботи та вибір реалізації

Мета — спроєктувати й описати UI/UX, навігацію та взаємодію користувача з клієнтською частиною застосунку: перегляд подій, календар, пошук, створення/редагування, категорії та налаштування. У межах практики розвивається Android-застосунок, створений у лабораторних № 1–5.

Обрано Flutter + Dart із Material 3. Інтерфейс у проєкті розподілений на компоненти AppShell, EventEditor, SearchPane, CategoryPane, SettingsPane та EventTile. Керування станом реалізоване через AppController (ChangeNotifier), збереження — через SqliteEventRepository.

Уточнення: Методичні вказівки № 6–7 орієнтовані на вебфронтенд (Vite/Next.js, HTML/CSS, REST API). У цій версії реалізовано саме Android UI на Flutter; ці вебтехнології та REST-виклики НЕ реалізовані. Погодження альтернативного технологічного стека з викладачем залишається необхідним.

2. Основні користувацькі сценарії

Код	Дія користувача	Результат в інтерфейсі
UC-01	Відкриття головної	Лічильники подій, список найближчих запланованих справ
UC-02	Перегляд календаря	Перемикання режимів день/тиждень/місяць; перегляд дня
UC-03	Створення події	Форма з назвою, часом, категорією, повторенням та нагадуваннями
UC-04	Редагування / видалення	Підтвердження; для серії — вибір окремого входження або серії
UC-05	Пошук	Пошук за текстом, фільтр категорії та діапазону дат
UC-06	Категорії	Список, створення, редагування і видалення категорій
UC-07	Налаштування	Теми, імпорт/експорт JSON, інформація про сповіщення

3. Карта навігації

У нижній панелі на вузькому екрані та в боковому меню на широкому екрані доступні п’ять основних розділів. Додавання та редагування відкривають діалогову форму; перегляд події — діалог деталей.

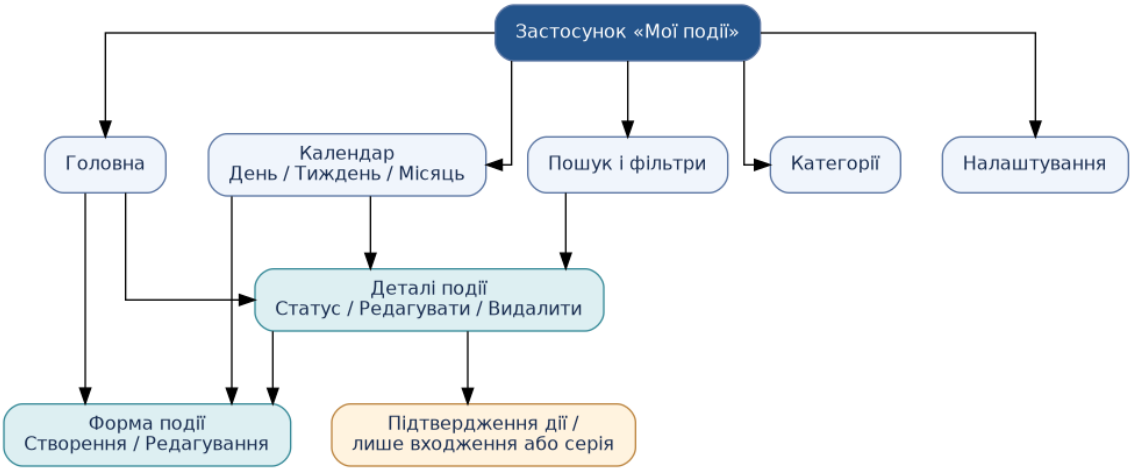


Рисунок 1 — Карта навігації застосунку

4. Wireframe основних екранів

До розробленої структури UI належать головний екран із найближчими подіями й форма введення. Наведені wireframe демонструють компонування, а не фактичний вигляд запущеної програми.

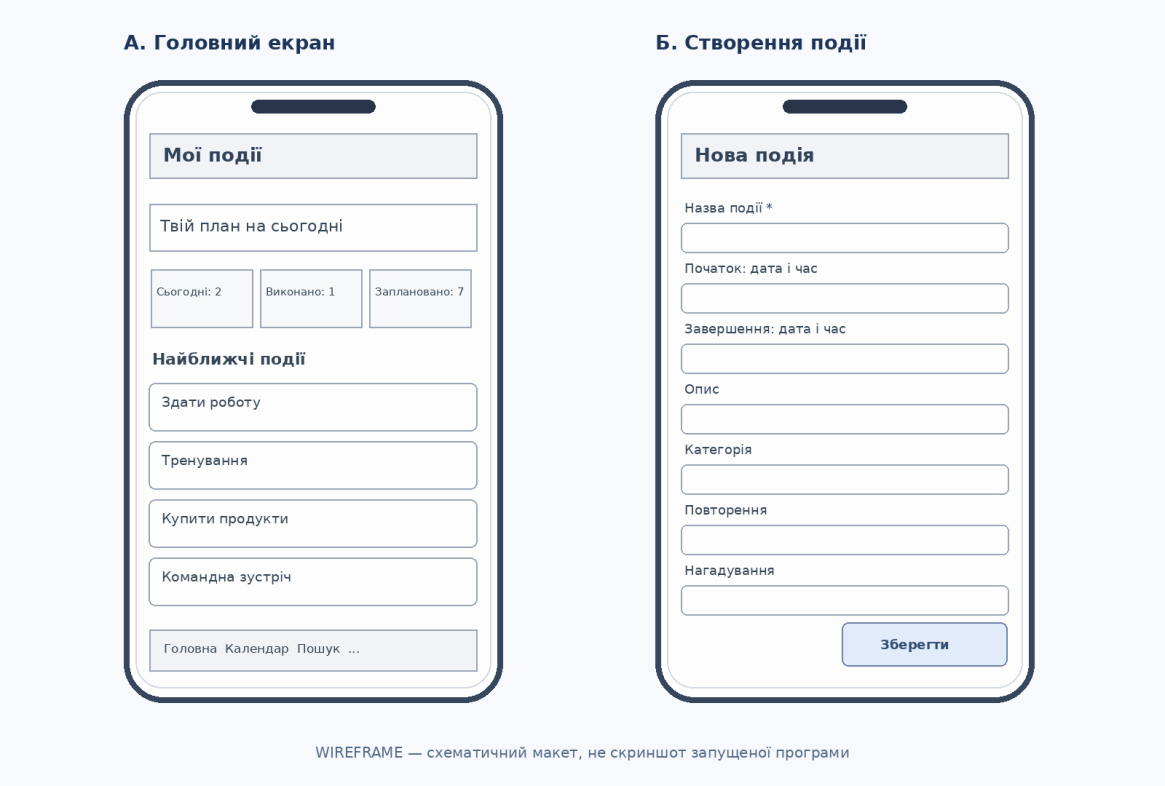


Рисунок 2 — Схематичні wireframe головної та форми події

5. Макети інтерфейсу і адаптивність

У коді AppShell використовується LayoutBuilder із порогом maxWidth >= 860. Для широкого екрана показується NavigationRail, для вузького — NavigationBar. Карти подій, поля введення, вибір категорії й системні діалоги створені віджетами Flutter.

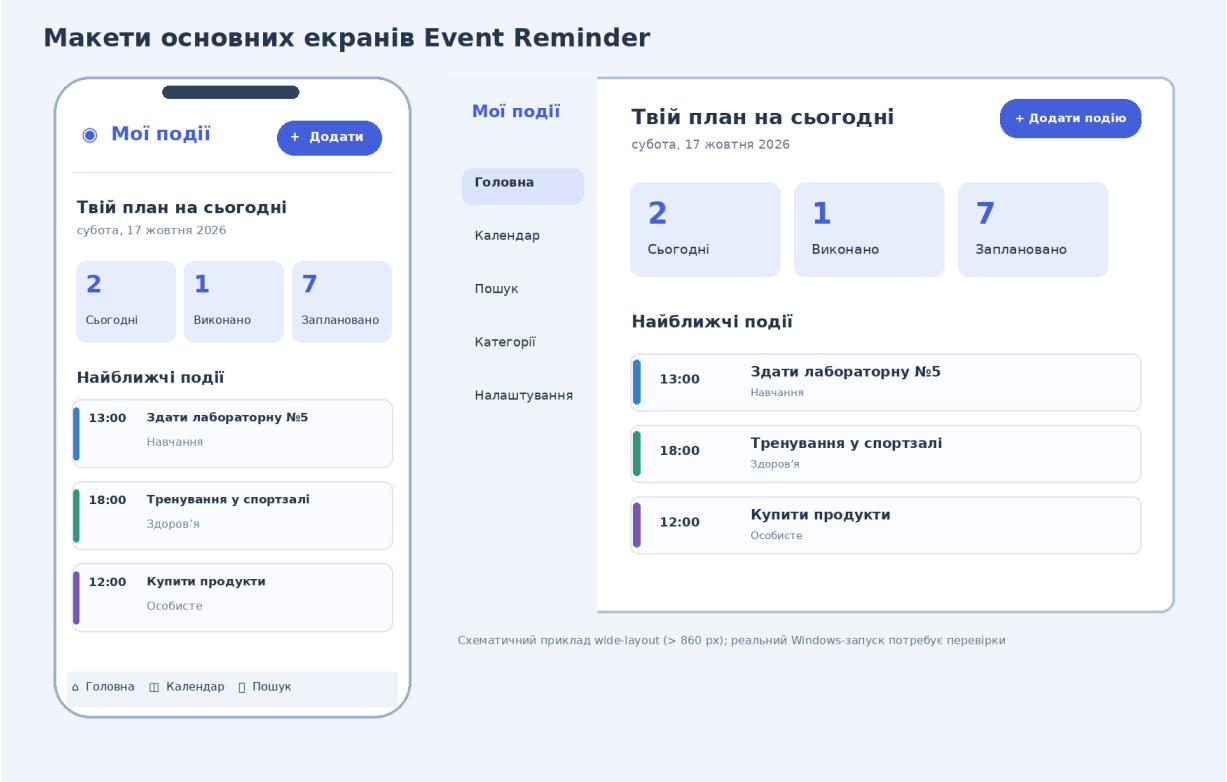


Рисунок 3 — Ілюстративні макети mobile і desktop (не фактичні скриншоти)

Уточнення: Desktop-макет відображає концептуальну широку компоновку з наявного Flutter-коду. Раніше Windows-збірка у користувача не проходила через плагін flutter_local_notifications_windows; тому реальний desktop-запуск і скриншот наразі не підтверджені.

6. Схема декомпозиції інтерфейсу

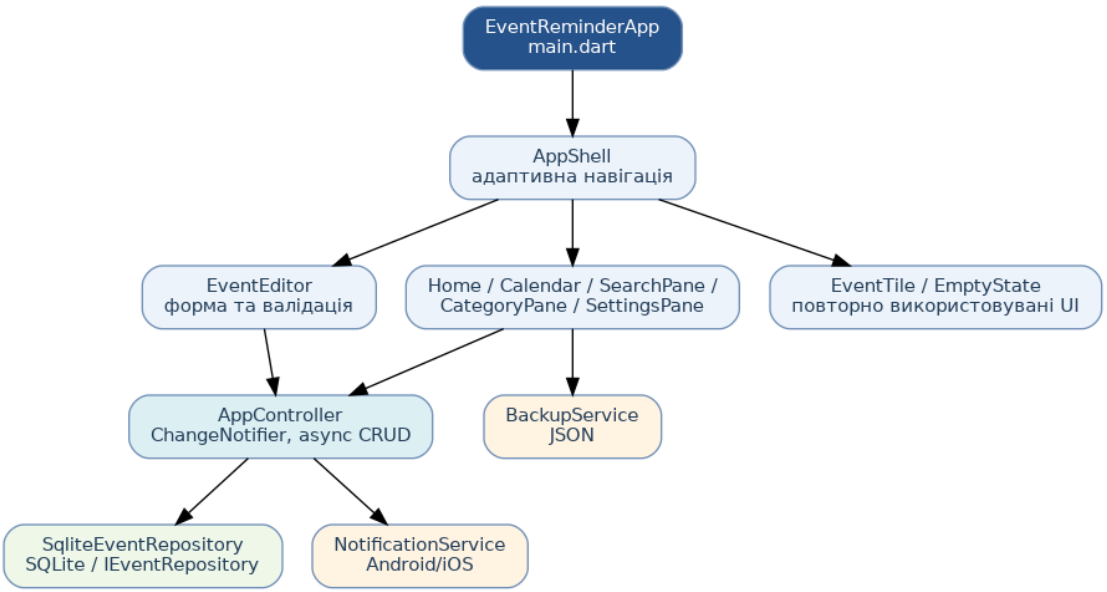


Рисунок 4 — Компонентна декомпозиція Flutter UI та шарів застосунку

7. Структура проєкту

Фрагмент дерева каталогів

```
event_reminder_complete/
├── android/                    # Android збірка та дозволи
├── ios/                        # iOS каркас
├── windows/                    # Windows каркас
├── lib/
│   ├── main.dart               # Створення MaterialApp
│   ├── presentation/
│   │   ├── app_shell.dart      # Навігація, головна, календар, пошук, категорії
│   │   ├── event_editor.dart   # Діалог створення / зміни події
│   │   └── ui_helpers.dart     # EventTile, EmptyState, повідомлення
│   ├── application/
│   │   └── app_controller.dart # ChangeNotifier, зміна стану
│   ├── domain/                 # Моделі й правила повторення
│   ├── data/                   # SQLite і репозиторій
│   └── infrastructure/         # Нагадування і резервні копії
├── test/                       # Тести повторень та сховища
└── pubspec.yaml                # Залежності
```

8. Фрагменти коду та пояснення

8.1. Адаптивна навігація — у AppShell логіка вибору нижньої панелі чи бокового меню залежить від ширини. Перемикання розділів оновлює стан StatefulWidget.

Скорочений і прокоментований фрагмент із lib/presentation/app_shell.dart

```
return LayoutBuilder(builder: (ctx, limits) {
  final wide = limits.maxWidth >= 860; // поріг desktop/mobile
  // Панель широкого екрана і нижня панель вузького екрана.
  return Scaffold(
    body: SafeArea(child: Row(children: [
      if (wide) NavigationRail(
        selectedIndex: tab,
        onDestinationSelected: (v) => setState(() => tab = v),
        destinations: [/* п’ять NavigationRailDestination */],
      ),
      Expanded(child: content),
    ])),
    bottomNavigationBar: wide ? null : NavigationBar(
      selectedIndex: tab,
      onDestinationSelected: (v) => setState(() => tab = v),
      destinations: [/* п’ять NavigationDestination */],
    ),
  );
});
```

8.2. Валідація — поле назви не може бути порожнім, допускається не більше 150 символів. У методі _save() перевіряються початок і завершення події, параметри повторення та інші умови.

Скорочений фрагмент із lib/presentation/event_editor.dart

```
TextFormField(
  controller: _title,
  decoration: const InputDecoration(labelText: 'Назва *'),
  validator: (v) => v == null || v.trim().isEmpty
      ? 'Введи назву' // повідомлення користувачу
      : v.trim().length > 150
          ? 'Максимум 150 символів'
          : null,
);
```

8.3. Асинхронна робота з даними та станом — після CRUD-операцій AppController завантажує оновлений знімок даних і викликає notifyListeners(). Це оновлює віджети через AnimatedBuilder.

Фрагмент із lib/application/app_controller.dart із пояснювальними коментарями

```
Future<void> save(EventItem item) async {
  if (item.id == null) {
    await repository.create(item); // запис нового Event у SQLite
  } else {
    await repository.update(item); // оновлення існуючого Event
  }
  await reload(); // перечитування та повідомлення UI
}
Future<void> reload() async {
  snapshot = await repository.loadAll();
  ready = true;
  notifyListeners(); // перерисовка підписаних екранів
  notificationWarning = await notifications.synchronize(snapshot);
  notifyListeners();
}
```

8.4. Інтеграція з даними — у чинній Android-версії запити виконуються до локальної SQLite через IEventRepository, а не до REST-сервера. Для буквального виконання пункту 9 методички потрібен окремий HTTP-шар/mock-server; у звіті це не подається як виконане.

9. Usability: аналіз інтерфейсу

Критерій	Рішення у проєкті	Що перевірити вручну
Зрозумілість	Назви вкладок і кнопка «Додати»	Користувач знаходить створення без підказки
Послідовність	Material 3, єдині картки й діалоги	Вигляд дій однаковий в усіх розділах
Валідація	Перевірка title і часових меж	Порожня назва / некоректний час показують помилку
Зворотний зв’язок	SnackBar, індикатор завантаження	Після збереження подія з’являється у списку
Контроль	Підтвердження видалення, вибір області серії	Можна скасувати руйнівну операцію
Адаптивність	NavigationRail / NavigationBar	Оцінити на телефоні й широкому екрані
Доступність	Стандартні Material-компоненти	Перевірити збільшення шрифтів і контраст

Тестовий висновок: структура інтерфейсу та захист від поширених помилок підтверджуються кодом. Суб’єктивну оцінку зручності і поведінку на різних розмірах екрана варто підтвердити коротким ручним тестуванням та скриншотами.

10. Скріншоти реалізованого інтерфейсу — додати після зйомки

Для фінального звіту методичка прямо вимагає скриншоти готового інтерфейсу на мобільному і десктопному екрані. Ілюстративні макети в розділі 5 їх не замінюють. У поточному пакеті фактичних скриншотів цих екранів немає.

№	Необхідний доказ	Як зняти
1	Головна сторінка Android	Відкрити «Мої події», зробити скриншот з подіями
2	Календар Android	Перейти «Календар» → «Місяць» і зробити скриншот
3	Форма події Android	Натиснути «Додати» і показати поля та валідацію
4	Пошук / категорії / налаштування Android	Зняти один або декілька додаткових екранів
5	Широкий layout / desktop	Запустити працездатну desktop-версію та зафіксувати NavigationRail

Рекомендований формат скріншотів: PNG. У Word додати їх із підписами «Рисунок 5 — ...», «Рисунок 6 — ...»; не підписувати макети як знімки працюючої програми.

11. Відповідність пунктам методичних вказівок

Пункт №6–7	Статус у Flutter-проєкті
1–4. Сценарії, екрани, wireframe, прототип переходів	Є код навігації + карта й макети у звіті
5. Vite/Next.js	Не застосовується до Flutter; не виконано буквально
6. Декомпозиція UI	AppShell, EventEditor, SearchPane, CategoryPane, SettingsPane, EventTile
7. Desktop + Mobile через CSS Flexbox/Grid	Є адаптивний Flutter LayoutBuilder, але не HTML/CSS; desktop-знімок відсутній
8. Обробка подій / валідація	Реалізовано у Flutter
9. Асинхронний REST CRUD	Не реалізовано; зараз асинхронні операції з локальною SQLite
10–11. Usability та обґрунтування	Описано в розділах 1, 5 і 9

12. Висновки

На базі попередніх лабораторних робіт проаналізовано та задокументовано Android-інтерфейс Event Reminder, реалізований на Flutter. Визначено основні сценарії використання, сформовано карту навігації, wireframe та макети основних екранів, показано компонентний поділ і приклади коду обробки стану, CRUD та валідації. Використання LayoutBuilder забезпечує різні схеми навігації для вузьких і широких екранів; дані зберігаються локально в SQLite.

Для завершення доказової частини необхідно додати фактичні скриншоти з Android та широкого екрана. Оскільки методичка орієнтована на React/Vite і REST API, прийнятність Flutter як альтернативної реалізації слід погодити з викладачем; інакше для буквального дотримання пунктів 5, 7 та 9 буде потрібна вебчастина.